



MongoDB



01 | Introduction



Qasim Nadeem | Principal Technologist, ICT Innovative

Meet Your Instructor – Qasim Nadeem

- Microsoft Certified Professional in Many Web and Cloud Technologies
- Started Professional Life in 1989-90
- Spend 7 years in Low Level Software Engineering
- A Decade in Networks i.e. MCSE CCNA CCNP
- Last Decade in Web & Software Engineering i.e. HTML5, PHP, Java, Android, ASP.NET, C#, SQL Server, SharePoint, MEAN Stack
- Running a Software House, Provides Consultancy to Ecommerce & Web Industry.
- Working in capacity of Principal Technologist & Head of Trainings

Introduction to Mongo



Course Topics

You've Got Documents! A MongoDB Jump Start

01 | Introduction to NoSQL Document Oriented Databases

02 | Running MongoDB on Local Machine

03 | MongoDB Database Organization

04 | MongoDB Terms

05 | Getting Started With Database & Collection [Create/Drop]

06 | MongoDB Datatypes

07 | MongoDB CRUD

08 | Indexes & Aggregation

09 | Mongo Server Replica

10 | Backup/Restore & Sharding

Storage Engine

The storage engine is the primary component of MongoDB responsible for managing data. MongoDB provides a variety of storage engines, allowing you to choose one most suited to your application.

WiredTiger Storage Engine (Default) is the default storage engine starting in MongoDB 3.2. It is well-suited for most workloads and is recommended for new deployments. WiredTiger provides a document-level concurrency model, checkpointing, and compression, among other features.

In MongoDB Enterprise, WiredTiger also supports Encryption at Rest. See [Encrypted Storage Engine](#).

In-Memory Storage Engine is available in MongoDB Enterprise. Rather than storing documents on-disk, it retains them in-memory for more predictable data latencies.

The **journal is a log** that helps the database recover in the event of a hard shutdown.

GridFS is a versatile storage system that is suited to handling large files, such as those exceeding the 16 MB document size limit.

Running MongoDB on Local Machine

The following binaries are installed onto your machine Install MongoDB

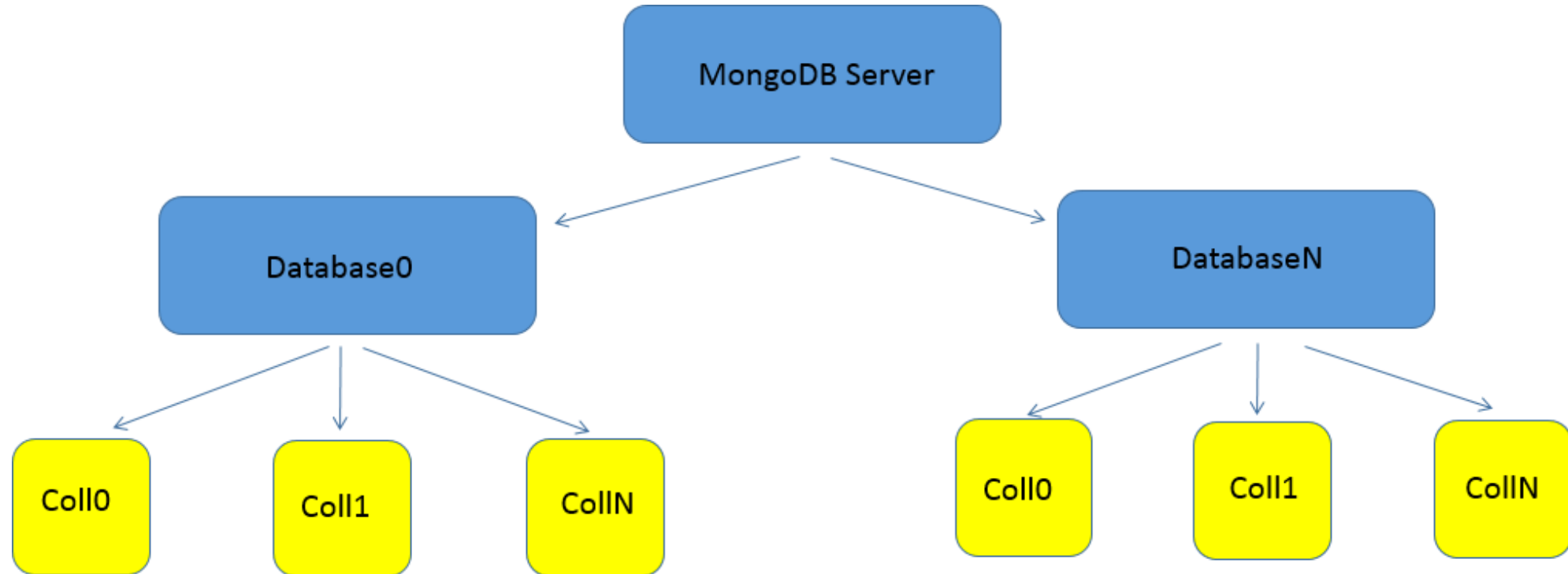
<https://www.mongodb.com/try/download/community>

<https://www.mongodb.com/try/download/shell>

mongod – The database server process

- Opens a CLI window and runs the above command
- mongosh – The mongodb CLI
 - Opens another CLI window and runs the the above command
- mongoimport – A data import utility
 - Load bulk data from CSV or JSON files

MongoDB Database Organization



MongoDB Terms

The following table shows the relationship of RDBMS terminology with MongoDB.

RDBMS	= MongoDB
Database	= Database
Table	= Collection
Tuple/Row	= Document
Column	= Field
Table Join	= Embedded Documents
Primary Key	= Primary Key (Default key <code>_id</code> provided by mongodb itself)
Database Server	= mongod
Client	= mongo
BSON	= BSON is a binary serialization format used to store documents and make remote procedure calls in MongoDB.
JSON in Mongo	= JSON can only represent a subset of the types supported by BSON. The following drivers use the Extended JSON v2.0: C/C++, Go, Java, Node, Perl, PHPC, Python, Scala

MongoDB Queries



Seek Help - db.help() & Check Stats - db.stats()

To get a list of commands, type db.help() in MongoDB client.

Example:

```
>db.help()
```

DB Statistics:

```
>db.stats()
```

Create Database – List DBs & Collections, Use

MongoDB use DATABASE_NAME is used to create database. The command will create a new database, if it doesn't exist otherwise it will return the existing database.

Example:

```
use DATABASE_NAME
```

Example:

```
>use Tech           //create & switched to db Tech
```

```
>db Tech //To check your currently selected database use the command db
```

```
>show dbs //gives databases list
```

```
> show collections //list all tables/collection in a database
```

Your created database (Tech) is not present in list. To display database you need to have a collection in it, so we are creating a collection by inserting a document.

```
>db.company.insert({"name":"interact"})
```

In mongodb default database is test. If you didn't create any database then collections will be stored in test database.

Drop Database – db.dropdatabase()

MongoDB `db.dropDatabase()` command is used to drop a existing database.

Example:

```
>db.dropDatabase()
```

This will delete the selected database. If you have not selected any database, then it will delete default 'test' database.

Note

First, check the list of available databases by using the command, `show dbs`.

Create Collection – db.createCollection()

MongoDB db.createCollection() command is used to create a collection [generally known as table].

Example: db.createCollection("name" [, "options are optional"])

```
>db.createCollection("company")
```

```
>db.createCollection("mycol", { capped : true, size : 6142800, max : 10000 } )
```

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. If you specify true, you need to specify size parameter also.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. If capped is true, then you need to specify this field also.
max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

List & Drop Collections

Example:

```
>show collections
```

List all the collection in selected database.

```
>db.CollectionName.drop()
```

Deletes the selected collection in current database.

Datatypes

MongoDB supports many datatypes. Some of them are –

- General Datatypes String [utf-8], Integer[32bit or 64bit], Boolean, Double, Date[d/m/y]
- Min/ Max keys – This type is used to compare a value against the lowest and highest BSON elements.
- Arrays – This type is used to store arrays or list or multiple values into one key.
- Timestamp – ctimestamp. This can be handy for recording when a document has been modified or added.
- Object – This datatype is used for embedded documents.
- Null – This type is used to store a Null value.
- Symbol – This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
- Object ID – This datatype is used to store the document's ID.
- Binary data – This datatype is used to store binary data.
- Code – This datatype is used to store JavaScript code into the document.
- Regular expression – This datatype is used to store regular expression.

Insert Document – db.insert()

To insert data into MongoDB collection, you need to use MongoDB's insert() or save() method.

Example:

```
>db.CollectionName.insert(document)
```

```
>db.company.insert({  
  _id: ObjectId(7df78ad8902c),  
  title: 'MongoDB Overview',  
  description: 'MongoDB is no sql database',  
  by: 'Tech trainings',  
  url: 'http://www.Tech-trainings.com',  
  tags: ['mongodb', 'database', 'NoSQL'],  
  likes: 100  
})
```

MongoDB unique 12digit object id:

_id: ObjectId(4 bytes timestamp, 3 bytes machine id, 2 bytes process id, 3 bytes incrementer)

Insert Document – insertOne(), insertMany()

Examples:

```
db.inventory.insertOne(  
  { item: "canvas", qty: 100, tags: ["cotton"], size: { h: 28, w: 35.5, uom: "cm" } }  
)
```

```
db.inventory.insertMany([  
  { item: "journal", qty: 25, tags: ["blank", "red"], size: { h: 14, w: 21, uom: "cm" } },  
  { item: "mat", qty: 85, tags: ["gray"], size: { h: 27.9, w: 35.5, uom: "cm" } },  
  { item: "mousepad", qty: 25, tags: ["gel", "blue"], size: { h: 19, w: 22.85, uom: "cm" } }  
])
```

Query Document – db.find()

To find/query data into MongoDB collection, you need to use MongoDB's find() method.

find() method will display all the documents in a non-structured way.
To display the results in a formatted way, you can use pretty() method.

Example:

```
>db.company.find() or db.inventory.find( {} ) equivalent to select * from company
```

```
>db.company.find().pretty()
```

```
>db.mycol.find({},{"title":1,_id:0}) for selective fields display use 1 or 0
```

```
>db.mycol.find({},{"title:1}).pretty();
```

```
>db.awards.find({name:{first:"James",last:"Gosling"}}).pretty();
```

```
>db.awards.find({"name.first":"James"}).pretty();
```

```
>db.awards.find({"name.first":/^J/}).pretty();
```

```
>db.inventory.find( { tags: ["red", "blank"] } ) querying array
```

Query Document – Clauses

To query the document on the basis of some condition, you can use following operations:

Equality {<key>:<value>} db.mycol.find({"by":"Tech"}).pretty()

Less Than {<key>:{\$lt:<value>}} db.mycol.find({"likes":{\$lt:50}}).pretty()

Less Than Equals {<key>:{\$lte:<value>}} db.mycol.find({"likes":{\$lte:50}}).pretty()

Greater Than {<key>:{\$gt:<value>}} db.mycol.find({"likes":{\$gt:50}}).pretty()

Greater Than Equals {<key>:{\$gte:<value>}} db.mycol.find({"likes":{\$gte:50}}).pretty()

Not Equals {<key>:{\$ne:<value>}} db.mycol.find({"likes":{\$ne:50}}).pretty()

And in MongoDB db.mycol.find({key1:value1, key2:value2}).pretty()

Or in MongoDB db.mycol.find({ \$or: [{key1: value1}, {key2:value2}] }).pretty()

And & Or db.mycol.find({"likes": {\$gt:10}, \$or: [{"by": "tutorials"}, {"title": "MongoDB Overview"}]}).pretty()

Query Document – Clauses Examples

Examples:

```
>db.mycol.find({$and:[{'url':http://www.Tech-trainings.com/mongodb/},{'likes':{$gt:100}}]}).pretty();
```

```
>db.mycol.find({$or:[{'url':http://www.Tech-trainings.com/mongodb/},{'likes':{$gt:100}}]}).pretty();
```

```
>db.inventory.find( { status: "A", $or: [ { qty: { $lt: 30 } }, { item: /^p/ } ] } ) //equivalent item LIKE "p%"
```

```
>db.inventory.find( { status: { $in: [ "A", "D" ] } } )
```

```
>db.inventory.find( { status: "A", qty: { $lt: 30 } } )
```

Projection – db.find(), limit(), skip(), sort()

In MongoDB, projection means selecting only the necessary data rather than selecting whole of the data of a document. eg. If a document has 5 fields and you need to show only 3, then select only 3 fields from them.

Example:

```
{ "_id" : ObjectId(5983548781331adf45ec5), "title":"MongoDB Overview"}  
{ "_id" : ObjectId(5983548781331adf45ec6), "title":"NoSQL Overview"}  
{ "_id" : ObjectId(5983548781331adf45ec7), "title":"Tech Trainings Overview"}
```

>db.mycol.find({},{"title":1,_id:0}) for selective fields display use 1 or 0

>db.mycol.find({},{"title":1}).pretty();

The limit() method limits the number of documents that displays when find execute.

>db.mycol.find({},{"title":1,_id:0}).limit(2)

The skip() method skip the number of documents specified.

>db.mycol.find({},{"title":1,_id:0}).limit(1).skip(1)

The sort() method skip the number of documents specified.

>db.mycol.find({},{"title":1,_id:0}).sort({"title":-1})

Update Document – db.update() & db.save()

MongoDB's update() and save() methods are used to update document into a collection.

The update() method updates the values in the existing document

Example: { "_id" : ObjectId(5983548781331adf45ec5), "title": "MongoDB Overview" }
{ "_id" : ObjectId(5983548781331adf45ec6), "title": "NoSQL Overview" }

```
>db.company.update({'title':'MongoDB Overview'},{$set:{'title':'New MongoDB Tutorial'}})
```

By default, MongoDB will update only a single document. To update multiple documents, you need to set a parameter 'multi' to true.

```
>db.mycol.update({'title':'MongoDB Overview'},{$set:{'title':'New MongoDB Tutorial'}},{multi:true})
```

The save() method replaces the existing document with the document passed in save() method.

Example:

```
>db.mycol.save({"_id" : ObjectId(5983548781331adf45ec7), "title": "Tech Trainings New Topic", "by": "Tech Trainings"})
```

Update Document –updateOne()

MongoDB's updateOne() methods are used to update single document into a collection.

```
db.inventory.updateOne(  
  { item: "paper" },  
  {  
    $set: { "size.uom": "cm", status: "P" },  
    $currentDate: { lastModified: true }  
  }  
)
```

uses the \$set operator to update the value of the size.uom field to "cm" and the value of the status field to "P",

uses the \$currentDate operator to update the value of the lastModified field to the current date.

If lastModified field does not exist, \$currentDate will create the field. See \$currentDate for details.

Update Document –updateMany()

MongoDB's updateOne() method used to update many documents in single go so following command updates all documents in inventory collection where qty is less than 50.

```
db.inventory.updateMany(  
  { "qty": { $lt: 50 } },  
  {  
    $set: { "size.uom": "in", status: "P" },  
    $currentDate: { lastModified: true }  
  }  
)
```

Update Document –replaceOne()

To replace the entire content of a document except for the `_id` field, pass an entirely new document as the second argument to `db.collection.replaceOne()`.

When replacing a document, the replacement document must consist of only field/value pairs; i.e. do not include update operators [`$min`, `$max`, `$mul` etc] expressions.

```
db.inventory.replaceOne(  
  { item: "paper" },  
  { item: "paper", instock: [ { warehouse: "A", qty: 60 }, { warehouse: "B", qty: 40 } ] }  
)
```

Remove Document – remove(), deleteOne() etc

MongoDB's remove() method is used to remove a document from the collection.

remove() method accepts two parameters. One is deletion criteria and second is flag value.

Example:

```
{ "_id" : ObjectId(5983548781331adf45ec5), "title":"MongoDB Overview"}  
{ "_id" : ObjectId(5983548781331adf45ec6), "title":"NoSQL Overview"}  
{ "_id" : ObjectId(5983548781331adf45ec7), "title":"MongoDB Overview"}
```

Remove all records

```
>db.mycol.remove({'title':'MongoDB Overview'},1)  
>db.inventory.deleteMany({})  
>db.inventory.deleteMany({ status : "A" })
```

Remove only one record

Example:

```
>db.mycol.remove({'title':'MongoDB Overview'},1)  
>db.inventory.deleteOne( { status: "D" } )
```

Creating Index – ensureIndex()/createIndex()

Indexes support the efficient resolution of queries. Without indexes, MongoDB must scan every document of a collection to select those documents that match the query statement.

ensureIndex is deprecated and now its just an alias for createIndex.

Value 1 is for ascending and -1 is for descending order.

```
>db.mycol.ensureIndex({"title":1})
```

```
>db.collection.createIndex( { name: -1 } )
```

Index on multiple fields:

```
>db.mycol.ensureIndex({"title":1,"description":-1})
```

list of all indexes on a collection, use the db.collection.getIndexes() method

Remove Specific Index

```
>db.accounts.dropIndex( { "tax-id": 1 } )
```

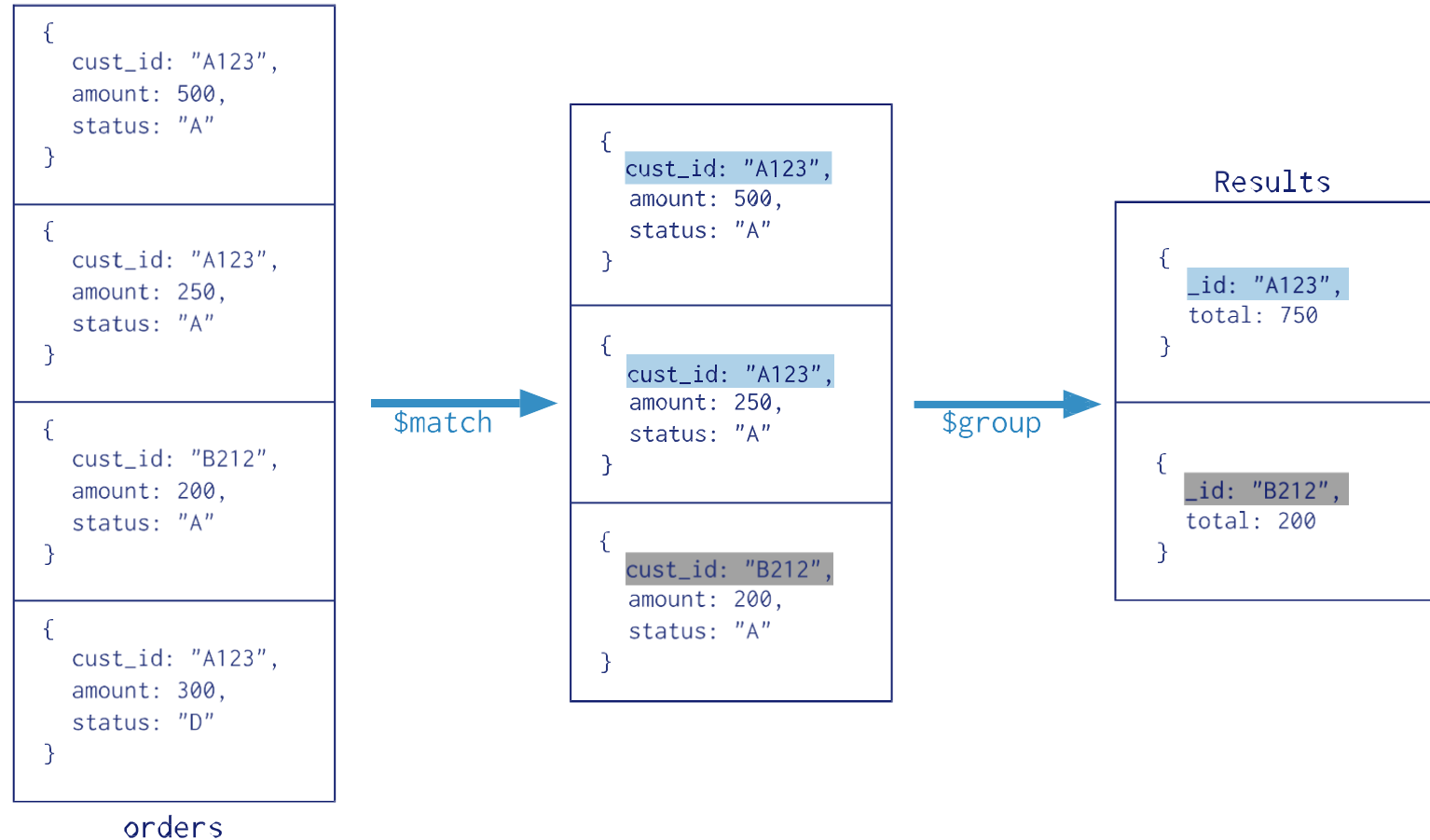
Drop all Indexes

```
>db.accounts.dropIndexes()
```

Aggregate

Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result.

Collection
↓
`db.orders.aggregate( [`
 `$match stage → { $match: { status: "A" } },`
 `$group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }`
 `]` `)`



Aggregate – aggregate()

Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result.

In SQL count(*) and with group by is an equivalent of mongodb aggregation.

Example:

In the collection you have following data:

```
{ _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview', description: 'MongoDB is no sql database', by_user: 'Tech trainings',
  url: 'http://www.Tech-trainings.com', tags: ['mongodb', 'database', 'NoSQL'], likes: 100
},{
  _id: ObjectId(7df78ad8902d)
  title: 'NoSQL Overview', description: 'No sql database is very fast', by_user: 'Tech
trainings', url: 'http://www.Tech-trainings.com', tags: ['mongodb', 'database', 'NoSQL'],
  likes: 10
},{
  _id: ObjectId(7df78ad8902e)
  title: 'Neo4j Overview', description: 'Neo4j is no sql database', by_user: 'Neo4j',
  url: 'http://www.neo4j.com', tags: ['neo4j', 'database', 'NoSQL'], likes: 750
},
```

```
> db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : 1}}}]
```

In the above example, we have grouped documents by field by_user and on each occurrence of by_user previous value of sum is incremented. Following is a list of available aggregation expressions.

Aggregate – aggregate()

Example:

```
> db.awards.aggregate([{$group : {_id : "$name", awards : {$sum : 1}}}]])
```

```
> db.awards.aggregate([{$group : {_id : "$contribs", awards : {$sum : 1}}}]])
```

```
> db.awards.aggregate([{$match:{contribs:"ALGOL"}},{$group : {_id : "$contribs", awards : {$sum : 1}}}]])
```

```
>db.orders.aggregate(  
  [  
    { $match: { status: "A" } },  
    { $group: { _id: "$cust_id", total: { $sum: "$amount" } } },  
    { $sort: { total: -1 } }  
  ],  
  {  
    explain: true  
  }  
)
```

//option explain to true to return information about the aggregation operation.

Aggregate – expressions

Expression	Description	Example
\$sum	Sums up the defined value from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$sum : "\$likes"}}}])
\$avg	Calculates the average of all given values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$avg : "\$likes"}}}])
\$min	Gets the minimum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$min : "\$likes"}}}])
\$max	Gets the maximum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$max : "\$likes"}}}])
\$push	Inserts the value to an array in the resulting document.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$push: "\$url"}}}])
\$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$addToSet : "\$url"}}}])
\$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "\$sort"-stage.	db.mycol.aggregate([{\$group : {_id : "\$by_user", first_url : {\$first : "\$url"}}}])
\$last	Gets the last document from the source documents according to the grouping. Typically this makes only sense together with some previously applied "\$sort"-stage.	db.mycol.aggregate([{\$group : {_id : "\$by_user", last_url : {\$last : "\$url"}}}])

MongoDB Administration for DBA's

© MongoDB, Inc. 2015. All rights reserved.

Replica

Replication is the process of synchronizing data across multiple servers.

Replication provides redundancy and increases data availability with multiple copies of data on different database servers.

Replication protects a database from the loss of a single server.

Replication also allows you to recover from hardware failure and service interruptions. With additional copies of the data, you can dedicate one to disaster recovery, reporting, or backup.

Advantages:

- To keep your data safe
- High (24*7) availability of data
- Disaster recovery
- No downtime for maintenance (like backups, index rebuilds, compaction)
- Read scaling (extra copies to read from)
- Replica set is transparent to the application

Replica in MongoDB

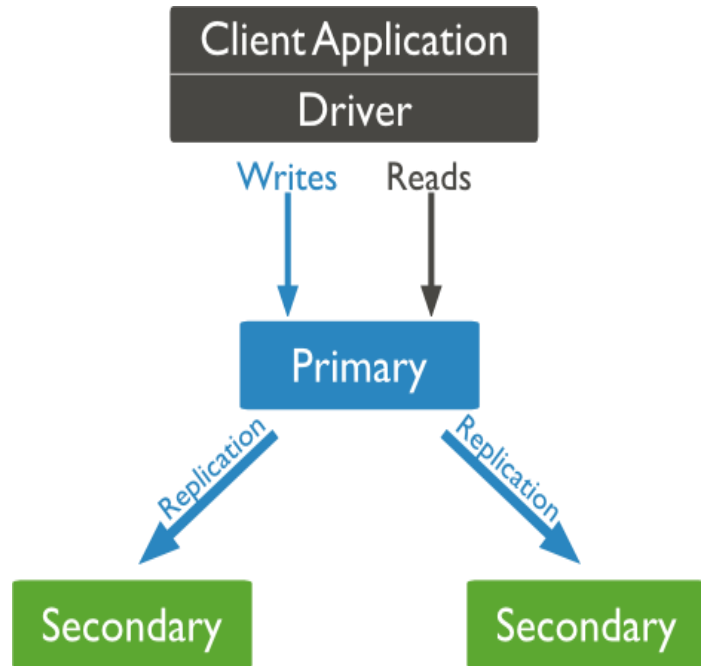
MongoDB achieves replication by the use of replica set.

A replica set is a group of mongod instances that host the same data set.

In a replica, one node is primary node that receives all write operations. All other instances, such as secondaries, apply operations from the primary so that they have the same data set.

- Replica set can have only one primary node.
- Replica set is a group of two or more nodes (generally minimum 3 nodes are required).
- In a replica set, one node is primary node and remaining nodes are secondary.
- All data replicates from primary to secondary node.
- At the time of automatic failover or maintenance, election establishes for primary and a new primary node is elected.
- After the recovery of failed node, it again join the replica set and works as a secondary node.

Replica in MongoDB



We'll convert standalone MongoDB instance to a replica set. To convert to replica set, following are the steps –

Shutdown already running MongoDB server.

Start the MongoDB server by specifying --replSet option.

Following is the basic syntax of --replSet –

```
mongod --port "PORT" --dbpath "YOUR_DB_DATA_PATH" --replSet "REPLICA_SET_INSTANCE_NAME"
```

Example

```
mongod --port 27017 --dbpath "c:\mongodb\data" --replSet rs0
```

It will start a mongod instance with the name rs0, on port 27017.

Now start the command prompt and connect to this mongod instance.

In Mongo client, issue the command **rs.initiate()** to initiate a new replica set.

To check the replica set configuration, issue the command **rs.conf()**. To check the status of replica set issue the command **rs.status()**.

db.adminCommand({ shutdown : 1}) issue at client if not connecting or accessing server so force shutdown the connections and restart

Add Members to Replica Set

To add members to replica set, start mongod instances on multiple machines. Now start a mongo client and issue a command `rs.add()`.

Syntax

The basic syntax of `rs.add()` command is as follows –

```
>rs.add(HOST_NAME:PORT)
```

Example

Suppose your mongod instance name is `mongod1.net` and it is running on port 27017. To add this instance to replica set, issue the command `rs.add()` in Mongo client.

```
>rs.add("mongod1.net:27017")
```

You can add mongod instance to replica set only when you are connected to primary node.

To check whether you are connected to primary or not, issue the command `db.isMaster()` in mongo client.

Data Backup – Dump MongoDB Data

To create backup of database in MongoDB, use **mongodump** command, will dump the entire data of your server into the dump directory.

Method:

Start your mongod server. Assuming that your mongod server is running on the localhost and port 27017, open a command prompt and go to the bin directory of your mongodb instance and type the command **mongodump**

Consider **mycol** is collection within default database **test**.

>**mongodump**

The command will connect to the server running at 127.0.0.1 and port 27017 and back all data of the server to directory **/bin/dump/**.

Following is a list of available options that can be used with the **mongodump** command.

Syntax	Description	Example
<code>mongodump /h /host:<hostname> /port:PORT_NUMBER</code>	This commmand will backup all databases of specified mongod instance.	<code>mongodump /host:<hostname> /port 27017</code>
<code>mongodump /out :BACKUP_DIRECTORY</code>	This command will backup only specified database at specified path.	<code>mongodump /out: c:/data/backup/</code>
<code>mongodump /collection: COLLECTION /d /db:<database-name></code>	This command will backup only specified collection of specified database.	<code>mongodump /collection: mycol /db:test</code>

ex> **mongodump /db:test /collection:techies /out:"c:/data/backup"**

Restore Backup

To restore backup data MongoDB's mongorestore command is used. This command restores all of the data from the backup directory.

Syntax

The basic syntax of mongorestore command is –

> mongorestore

connection options:

/h, /host:<hostname> mongodb host to connect to (setname/host1,host-2 for replica sets)
/port:<port> server port (can also use --host hostname:port)
--db <dbase_name> database to use when restoring from a BSON file
--collection <collection-name> collection to use when restoring from a BSON file
/dir:<directory-name> input directory

ex> mongorestore --db test --collection techies /dir: "/data/backup/test/techies.bson"

Sharding

Sharding is the process of storing data records across multiple machines and it is MongoDB's approach to meeting the demands of data growth.

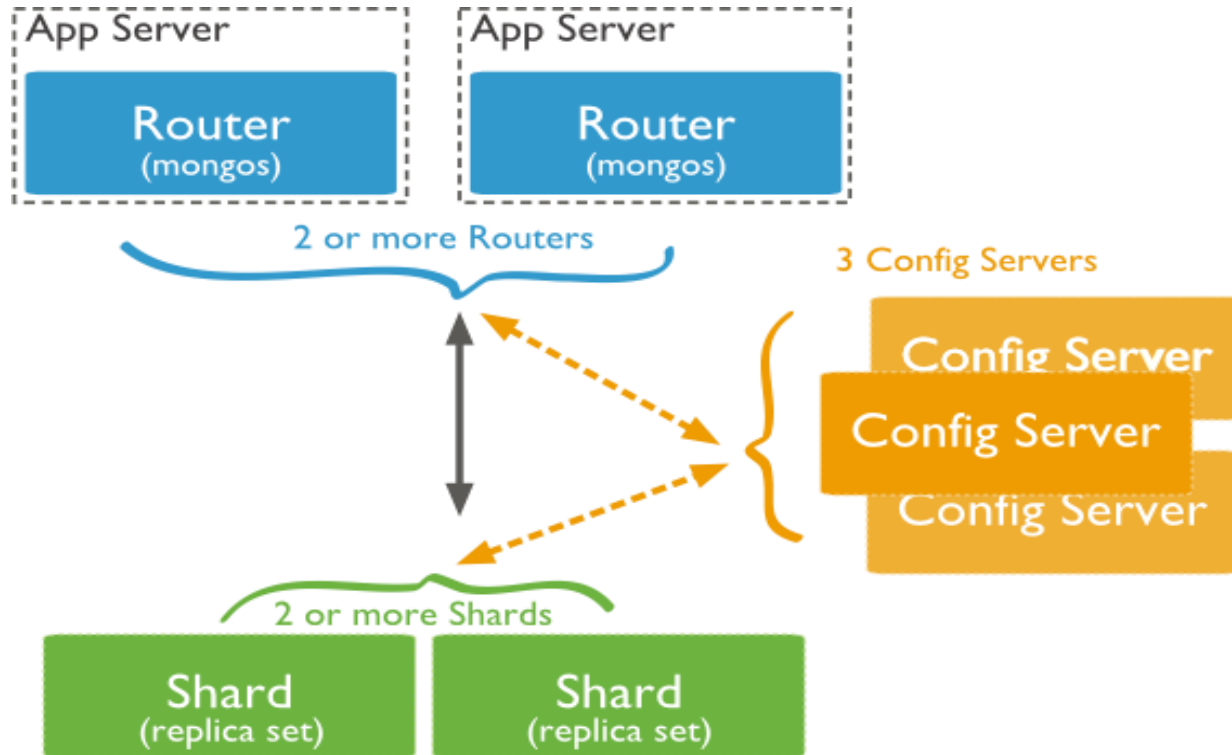
As the size of the data increases, a single machine may not be sufficient to store the data nor provide an acceptable read and write throughput. Sharding solves the problem with horizontal scaling.

In sharding, you add more machines to support data growth and the demands of read and write operations.

Why Sharding and What's not in Replica:

- In replication, all writes go to master node
- Latency sensitive queries still go to master
- Single replica set has limitation of 12 nodes
- Memory can't be large enough when active dataset is big
- Local disk is not big enough
- Vertical scaling is too expensive

Sharding



Shards – Shards are used to store data. They provide high availability and data consistency. In production environment, each shard is a separate replica set.

Config Servers – Config servers store the cluster's metadata. This data contains a mapping of the cluster's data set to the shards.

The query router uses this metadata to target operations to specific shards. In production environment, sharded clusters have exactly 3 config servers.

Query Routers – Query routers are basically mongo instances, interface with client applications and direct operations to the appropriate shard.

The query router processes and targets the operations to shards and then returns results to the clients.

A sharded cluster can contain more than one query router to divide the client request load. A client sends requests to one query router. Generally, a sharded cluster have many query routers.

Mongostat & Mongotop

Mongostat checks the status of all running mongod instances and return counters of database operations.

These counters include inserts, queries, updates, deletes, and cursors.

Command also shows when you're hitting page faults, and showcase your lock percentage. This means that you're running low on memory, hitting write capacity or have some performance issue.

To run the command, start your mongod instance. In another command prompt, go to bin directory of your mongodb installation and type mongostat.

```
d:\set up\mongodb\bin>mongostat
```

Mongotop tracks and reports the read and write activity of MongoDB instance on a collection basis.

By default, mongotop returns information in each second, which you can change it accordingly.

You should check that this read and write activity matches your application intention, and you're not firing too many writes to the database at a time, reading too frequently from a disk, or are exceeding your working set size.

To run the command, start your mongod instance. In another command prompt, go to bin directory of your mongodb installation and type mongotop.

```
d:\set up\mongodb\bin>mongotop
```

Development using MongoDB & PHP

Accessing Mongo DB within VS Code

- mongod –
 - Opens a CLI window and runs the above command
- VSCode:
 - Add mongodb for VSCode plugin
 - Connect using connections string: mongodb://localhost/mydb
 - Now from the left pane navigate your db collections & documents

DEMO

MongoDB – PHP Integration Example

MongoDB-PHP Establish Connection

To make a connection, you specify the database name, if it doesn't exist then MongoDB creates it automatically.

Step-1: download extension php_mongo.dll from <http://pecl.php.net/package/mongo/1.6.16/windows> or <https://windows.php.net/downloads/pecl/releases/mongo> and copy it to php ext folder on web server.

Latest extension is php_mongodb.dll which works with php version 7.

For full support of Mongo features use php library of mongodb.

Step-2: Add extension to php.ini as extension=php_mongo.dll

Following is the code snippet to connect to the database –

```
<?php
// connect to mongodb
$m = new MongoClient();
echo "Connection to database successfully";
// select a database
$db = $m->mydb;
echo "Database mydb selected";
?>
```

MongoDB-PHP Create Collection

Following is the code snippet to create a collection –

```
<?php
```

```
// connect to mongodb
```

```
$m = new MongoClient();
```

```
echo "Connection to database successfully";
```

```
// select a database
```

```
$db = $m->mydb;
```

```
echo "Database mydb selected";
```

```
$collection = $db->createCollection("mycol");
```

```
echo "Collection created successfully";
```

```
?>
```

MongoDB-PHP Insert Document [Record]

Following is the code snippet to insert a document –

```
<?php                                     $document = array(
    // connect to mongodb                 "title" => "MongoDB",
    $m = new MongoClient();              "description" => "database",
    echo "Connection to database successfully";    "likes" => 100,
                                           "url" => "http://www.ict-trainings.com/mongodb/",
                                           "by", "ict trainings"
                                           );
    // select a database
    $db = $m->mydb;
    echo "Database mydb selected";
    $collection = $db->mycol;
    echo "Collection selected successfully";
    $collection->insert($document);
    echo "Document inserted successfully";
    ?>
```

MongoDB-PHP Find all Documents

Following is the code snippet to select all documents –

```
<?php
// connect to mongodb
$m = new MongoClient();
echo "Connection to database successfully";

// select a database
$db = $m->mydb;
echo "Database mydb selected";
$collection = $db->mycol;
echo "Collection selected successfully";

$cursor = $collection->find();
// iterate cursor to display title of documents

foreach ($cursor as $document) {
    echo $document["title"] . "\n";
}
?>
```

MongoDB-PHP Update Document

Following is the code snippet to update a document –

```
<?php
// connect to mongodb
$m = new MongoClient();
echo "Connection to database successfully";

// select a database
$db = $m->mydb;
echo "Database mydb selected";
$collection = $db->mycol;
echo "Collection selected successfully";

// now update the document
$collection->update(array("title" => "MongoDB"),
    array('$set' => array("title" => "MongoDB Learning")));
echo "Document updated successfully";
// now display the updated document
$cursor = $collection->find();
// iterate cursor to display title of documents
echo "Updated document";
foreach ($cursor as $document) {
    echo $document["title"] . "\n";
}
?>
```

MongoDB-PHP Delete Document

Following is the code snippet to delete a document –

```
<?php
// connect to mongodb
$m = new MongoClient();
echo "Connection to database successfully";

// select a database
$db = $m->mydb;
echo "Database mydb selected";
$collection = $db->mycol;
echo "Collection selected successfully";

// now remove the document
$collection->remove(array("title"=>"MongoDB
Learning"),false);
echo "Documents deleted successfully";
// now display the available documents
$cursor = $collection->find();

// iterate cursor to display title of documents
echo "Updated document";
foreach ($cursor as $document) {
    echo $document["title"] . "\n";
}
?>
```

MongoDB Market



MongoDB in Local and International Market?

Pakistani Market:

<https://www.rozee.pk>

<https://www.jobee.pk>

International Market:

<https://www.careerone.com.au>

<https://www.bayt.com>

<https://www.upwork.com/o/jobs/browse/?q=html5>

And many more...

Learning Sources:

<https://docs.mongodb.com>

<https://university.mongodb.com>

